Wi-Fi CSI Human Activity Recognition Project

Plain-English Guide: What Is Done & How Every Part Works

A clear, easy-to-read walkthrough of the entire project, the 4 experiments, and every file

Repository: github.com/Armanmund/WIFI_CSI | Rooms: A1, A2, A3 | Activities: Walk, Stand, Jump, Empty

1. What This Project Is About (In Plain English)

Imagine a normal Wi-Fi router in a room sending invisible radio waves to a receiver. When a person walks, stands, or jumps in that room, their body bounces and disturbs those Wi-Fi waves. By reading those radio wave disturbances (called Wi-Fi CSI, or Channel State Information), an AI model can tell what the person is doing without needing any cameras or wearable sensors.

The Problem: Wi-Fi waves also bounce off the walls, furniture, and doors of that specific room. So if you train an AI model in Room A1, it learns the layout of Room A1 along with the human movements. When you take that exact same AI model and place it in a new room (Room A2 or Room A3), its accuracy drops sharply because the walls and furniture are different. Collecting thousands of new recordings for every single house or hospital room is too slow and expensive.

The Goal of This Project: Instead of collecting thousands of recordings in the new room, what if we only collect a tiny handful of recordings (for example, 1, 5, 10, or 20 short clips per activity) and quickly adjust ('fine-tune') our original AI model? This project measures exactly how many recordings from a new room are needed to bring the AI back to full accuracy.

The 5 Main Goals of Your Project (Saved in PROJECT_OBJECTIVES.md)

Goal 1 (Train in Room A1): Teach the AI model to recognize 4 activities (Walking, Standing, Jumping, Empty Room) in the first room (Room A1) and measure its best accuracy.

Goal 2 (Test in Unseen Rooms A2 & A3): Take the Room A1 model and test it directly in Room A2 and Room A3 with zero adjustments to see how much accuracy is lost.

Goal 3 (Quick Adaptation with Small Data): Give the Room A1 model a small number of recordings from the new room (1, 2, 3, 5, 10, 15, 20, 30, or 50 clips per activity) and measure how quickly its accuracy recovers.

Goal 4 (Compare Against Starting from Zero): Train a brand-new, blank AI model using those exact same small recordings to prove that starting from the Room A1 model is much better than starting from scratch.

Goal 5 (Create the Final Cost Graph): Plot a clear graph showing 'Accuracy vs. Number of New Recordings Needed' so anyone deploying Wi-Fi sensing knows the exact data cost for a new room.

2. What Is Done Right Now vs. What Is Left To Do

Think of this project like baking a cake: the entire kitchen, recipe book, oven timers, and cake molds (100% of the Python code) are completely built and ready. The only thing not done yet is buying the ingredients (downloading the dataset) and turning on the oven (running the training script).

Part of the Project	File Name	Status	Plain-English Explanation of What Is Built
Project Goals & Abstract	BTech_Abstract_CSI.pdf PROJECT_OBJECTIVES.md	DONE (100%)	Your official B.Tech abstract and the 5 project goals are written and saved in the project folder.
Control Panel (Settings)	configs/config.py	DONE (100%)	Stores all settings in one place: room names (A1, A2, A3), the 4 activities, training speed, and sample counts.
Dataset Downloader	download_data.py	DONE (100%)	Automatically downloads the Wi-Fi recordings from Google Drive and checks that all folders are present.

Part of the Project	File Name	Status	Plain-English Explanation of What Is Built
Data Reader & Organizer	src/data_loader.py	DONE (100%)	Opens the Wi-Fi recording files (.mat files), labels them (Walk=0, Empty=1, Jump=2, Stand=3), and picks small random samples.
The AI Brain (CNN Model)	src/model.py	DONE (100%)	Builds the 4-layer image-style AI network (CSIActivityCNN) that reads Wi-Fi signals like a 2D picture.
The Teacher (Trainer)	src/trainer.py	DONE (100%)	Teaches the AI model step-by-step, grades its accuracy, stops early if it stops improving, and saves the best version.
The 4 Experiments	src/experiments.py run_experiments.py	DONE (100%)	Runs all 4 experiments automatically from start to finish and collects all accuracy numbers.
Graph & Table Maker	src/visualisation.py src/utils.py	DONE (100%)	Draws all the final graphs (PDF and PNG pictures) and creates the final Excel-style CSV results table.
Practice Test (Smoke Test)	test_pipeline.py	DONE (100%)	Creates fake practice Wi-Fi files to test that all the code works without waiting for the real download.
Real Data & Final Graphs	few_shot_datasets/ results/ & models/	NOT RUN YET	We still need to install the Python packages, download the real Wi-Fi dataset, and run the program to get the final graphs.

3. Detailed Explanation of Every Part (In Simple Terms)

Part A: The Control Panel (configs/config.py)

Instead of scattering numbers and settings across dozens of files, configs/config.py acts as the single control panel for the entire project. It is divided into 4 simple sections:

- **1. Data Settings (DataConfig):** Tells the program which dataset folder to use (m1c4_PCA_80_300_extracted_3x4), which room is the starting room (A1), which rooms are the new test rooms (A2 and A3), and assigns a number to each of the 4 activities: Walk = 0, Empty Room = 1, Jump = 2, Stand = 3.
- **2. Model Settings (ModelConfig):** Tells the program to use a Convolutional Neural Network (CNN) with 4 output choices (one for each activity) and a 30% 'dropout' rate (a technique that randomly turns off 30% of neurons during training so the AI learns general patterns instead of memorizing).
- **3. Main Training Settings (TrainConfig):** Sets the rules for training the baseline model in Room A1: train for up to 200 rounds (epochs), look at 16 recordings at a time (batch size = 16), hold back 15% of Room A1 data as a practice quiz (validation split), and stop early if the quiz score does not improve for 20 rounds in a row.
- **4. Fine-Tuning Settings (FineTuneConfig):** Sets the rules for adapting to Room A2 and A3: test 9 different sample sizes (k = 1, 2, 3, 5, 10, 15, 20, 30, and 50 recordings per activity). For every sample size, repeat the test 25 times (5 random seeds x 5 random sample picks) so the final average is trustworthy and not a lucky guess.

Part B: Getting & Reading the Wi-Fi Data (download_data.py & src/data_loader.py)

Raw Wi-Fi signals are stored as MATLAB (.mat) files from the public ReWiS dataset. Each recording is a grid of numbers with 80 rows (representing 80 cleaned Wi-Fi frequency channels) and 300 columns (representing 300 moments in time). You can think of each recording as an 80 x 300 grayscale image of how the Wi-Fi signal rippled over time.

- **Downloading & Checking (download_data.py):** Connects to Google Drive and downloads the dataset into few_shot_datasets/. Then its verify_dataset() function checks all 12 folders (3 rooms x 4 activities) and counts how many .mat files are inside each folder to make sure nothing is missing.
- **Reading Each File (read_mat & load_environment_data):** Opens every .mat file in a room (such as train_A1/walk/), pulls out the 80 x 300 number grid, and attaches the matching activity label (0, 1, 2, or 3).

- **Splitting Data Fairly (create_dataloaders):** Splits the recordings into a 'study set' (training) and a 'quiz set' (validation), making sure both sets have an equal mix of walking, standing, jumping, and empty room clips.
- **Picking Small Samples (sample_k_shot):** When we test how the AI adapts with only a few recordings (say k = 5), this function randomly picks exactly 5 walking, 5 standing, 5 jumping, and 5 empty room recordings (20 clips total) from the new room.

Part C: The AI Brain (src/model.py - CSIActivityCNN)

Because each Wi-Fi recording is shaped like an 80 x 300 grid, src/model.py uses a 2D Convolutional Neural Network (CSIActivityCNN) - the same type of AI used to recognize objects in photos. It has 785,028 tunable weights (parameters), which is small enough to train quickly on a normal computer and small enough that it will not easily memorize tiny datasets.

Stage of the AI Brain	What It Does in Plain English	Size of the Signal Grid
Input Recording	Takes in one Wi-Fi recording as a single-layer 80 x 300 grid of numbers.	1 grid of 80 x 300
Filter Block 1	Scans the grid with 64 small 3x3 filters to spot simple ripples and sudden changes in the Wi-Fi wave, then shrinks the grid in half.	64 grids of 40 x 150
Filter Block 2	Combines those simple ripples into short motion patterns (like a footstep or body sway) and shrinks the grid in half again.	64 grids of 20 x 75
Filter Block 3	Increases to 128 filters to detect larger movement patterns across time and frequency, shrinking the grid once more.	128 grids of 10 x 37
Filter Block 4	Summarizes the entire recording into a compact 4 x 4 summary grid for each of the 128 filters (2,048 numbers total).	128 grids of 4 x 4 (2,048 numbers)
Final Decision Maker (Classifier Head)	Takes those 2,048 summary numbers, narrows them down to 256 key features, and outputs 4 final scores: one for Walk, Empty, Jump, and Stand. The highest score wins.	4 final scores (Walk, Empty, Jump, Stand)

Part D: The Teacher & Scorekeeper (src/trainer.py)

The file src/trainer.py is the engine that actually trains and tests the AI brain:

- **1. Making Results Repeatable (set_seed):** Locks the computer's random number generator so that if you run the experiment today and again next month, you get the exact same results.
- **2. Teaching Round-by-Round (train_epoch & validate):** In each round (epoch), the AI guesses the activity for a batch of recordings, checks how wrong its guess was (loss), and nudges its internal weights to do better next time. After each round, it takes a practice quiz on held-out validation data.
- **3. Smart Early Stopping (fit):** Every time the AI gets a new personal-best score on the practice quiz, the trainer saves a copy of the AI's weights. If the AI goes 20 rounds without beating its best quiz score, the trainer stops automatically and reloads the best saved version so the AI never over-trains.
- **4. Gentle Fine-Tuning (fine_tune):** When adapting the Room A1 model to a new room (A2 or A3), this function lowers the learning speed by 10 times (from 0.001 down to 0.0001). This ensures the AI gently adjusts to the new room without erasing what it already learned in Room A1.

4. How the 4 Experiments Work Step-by-Step (src/experiments.py)

When you run python run_experiments.py, the file src/experiments.py runs four scientific experiments in order. Here is what each experiment does in plain English:

Experiment	What We Do	Why We Do It	What Files It Creates
Experiment 1: Baseline in Room A1	We train the AI on 85% of the recordings from Room A1 and test it on the remaining 15% of Room A1.	Shows how accurate the AI is when tested in the exact same room where it was trained (our 'gold standard' target score).	- Saved AI Model: models/baseline_A1.pt - Training & Accuracy Graph - Room A1 Confusion Matrix

Experiment	What We Do	Why We Do It	What Files It Creates
Experiment 2: Zero-Shot Test in A2 & A3	We take the trained Room A1 model and test it directly on all recordings from Room A2 and Room A3 without any retraining.	Proves our main thesis problem: moving an AI to a new room causes a big drop in accuracy because the room layout changed.	- Accuracy drop numbers: results/cross_env_results.json - Confusion Matrices for Room A2 and Room A3
Experiment 3: Few-Shot Fine-Tuning	We take the Room A1 model and give it just k recordings per activity from the new room (testing k = 1, 2, 3, 5, 10, 15, 20, 30, 50). We fine-tune it gently and test it on the rest of the new room. We repeat this 25 times for every k.	Answers the core question of your project: 'How many labelled recordings from a new room do we need to recover 90% of our original accuracy?'	- Detailed scores for every k: finetuning_A2_results.json finetuning_A3_results.json - Average accuracy & 95% confidence range
Experiment 4: Training from Scratch (Control)	We repeat Experiment 3 using the exact same small sets of k recordings (1 to 50), except we start with a completely blank AI model instead of the Room A1 model.	Proves that pre-training in Room A1 actually helps! With only 5 or 10 samples, a fine-tuned model performs much better than a blank model trained from scratch.	- Control scores for every k: scratch_A2_results.json scratch_A3_results.json

5. The Final Graphs & Tables Created Automatically (src/visualisation.py)

Once the 4 experiments finish, src/visualisation.py and src/experiments.py automatically draw publication-ready figures (saved as both high-resolution PNG images and PDF files inside results/plots/) and a master table:

- **1. Baseline Training Curves (baseline/training_curves.png):** Two side-by-side line graphs showing the AI's error rate dropping and its accuracy climbing during Room A1 training, with a vertical dotted line marking the best round.
- **2. Confusion Matrices (confusion_matrix.png):** A 4x4 grid heatmap showing which activities the AI gets right and which ones it confuses (for example, whether it ever confuses 'Standing' with 'Empty Room').
- **3. Cross-Environment Bar Chart (summary/cross_env_comparison.png):** A clean bar chart comparing the high accuracy in Room A1 against the dropped zero-shot accuracy in Room A2 and Room A3.
- **4. The Main Deliverable - Fine-Tuning Curve (summary/finetuning_curve_A2.png & A3.png):** This is the most important figure for your B.Tech report. Along the bottom (x-axis) is the number of recordings per activity (1 to 50). Going up (y-axis) is the accuracy. It plots two lines with shaded confidence bands: the Fine-Tuned Model vs. the Trained-From-Scratch Model, plus a red star marking the exact number of samples where the model recovers 90% of its baseline accuracy.
- **5. Percentage Recovery Curve (summary/recovery_curve_A2.png):** Shows what percentage of the lost accuracy is recovered as you add more samples, with horizontal lines marking 50%, 75%, 90%, and 95% recovery.
- **6. Master Results Table (results/tables/summary_results.csv):** A spreadsheet file listing every experiment, environment, sample count k, average accuracy, standard deviation, and 95% confidence interval, ready to paste directly into your thesis report.

6. Next Steps: How to Run the Project on Your Computer

4 Simple Steps to Run the Experiments and Generate All Graphs

Step 1 - Install the required Python libraries: Open your terminal in the wifi_csi folder, create a virtual environment, and run: pip install -r requirements.txt

Step 2 - Test the code with fake practice data (takes ~1 minute): Run: python test_pipeline.py (This creates tiny practice files and tests all 4 experiments to confirm everything works.)

Step 3 - Download the real ReWiS Wi-Fi dataset: Run: python download_data.py (Or download the folder m1c4_PCA_80_300_extracted_3x4 from the Google Drive link in README.md and place it inside few_shot_datasets/).

Step 4 - Run the real experiments: Run: python run_experiments.py --quick (for a faster run) or python run_experiments.py (for the full 25-run statistical benchmark).